

Parameterized Synchronous FIFO threshold control with latency optimization by using First-word Fall –Through (FWFT)

¹Mr. PINNINTI KRANTHI, ²Mr. N SESHU KUMAR, ³Mr. K APPALA RAMU

¹PG Scholar, ²ASSOCIATE PROFESSOR, ³ASSOCIATE PROFESSOR

Department of Electronics and Communication Engineering

Miracle Educational Society Group of Institutions, Bhogapuram-535216, AP, India

Published online: XX March XXXX

Abstract –First-In First-Out (FIFO) memory structures play a vital role in modern digital systems by providing temporary data storage and synchronization between modules operating at different processing rates. This thesis is proposed with latency optimization through threshold control for FIFO is designed by using FIRST WORD FALL THROUGH(FWFT).Almost_full parameter is used to read data even the read enable is not high.The data is read immediately when the data is written after the write pointer exceeds threshold value.The proposed FIFO architecture supports configurable data width and memory depth, enabling scalability and flexibility for various FPGA-based applications using FWFT mode. The FIFO design incorporates dedicated read and writes pointer management, full and empty status flag generation.Synthesis and implementation results demonstrate successful FIFO operation with efficient resource utilization and reliable data transfer.

Index Terms – FIFO , FWFT,FIFO latency, FIFO threshold

I. INTRODUCTION

This design reduces read latency to **zero clock cycles** when the FIFO is empty. When data is written to an empty FIFO, it bypasses the internal memory array using a combinational multiplexer, updating data_out and dropping the empty flag on the **same clock cycle**.

Here is the optimized **First-Word Fall-Through (FWFT) Synchronous FIFO** updated to include a programmable **Almost-Full (AF)** threshold flag. Here is the complete **First-Word Fall-Through (FWFT)** mechanics and the look-ahead **Almost-Full (AF)** flag behaviors.

fifo_mem: A standard memory array structure (registers or synthesized RAM) that stores data packets sequentially.

wr_ptr and rd_ptr: These pointers are configured with **one extra bit** beyond the memory address space (e.g., 5 bits for a depth of 16). This extra bit acts as a **wrap-around bit** to distinguish between a completely empty FIFO and a completely full FIFO

empty: Asserted combinationally if both pointers match exactly, meaning no unread data remains.

full: Asserted if the lower address bits match but the top wrap-around bits are opposite, meaning the write pointer has looped around the memory space and caught up to the read pointer.

In hardware design, a **First-Word Fall-Through (FWFT) FIFO** is a memory variant where the oldest data automatically appears on the output bus as soon as the FIFO is no longer empty.

It acts like a "look-ahead" buffer, making data available immediately without requiring a read-enable signal.

II. RELATED WORK

Parameterized FIFO designs improve reusability, allowing a single architecture to serve multiple applications. FPGA vendors such as Xilinx and Intel provide dedicated memory resources and optimization techniques (e.g., distributed RAM inference) that facilitate efficient FIFO implementation.

Automatic Output: In a standard FIFO, you must assert the read-enable (RDEN) signal to see data. In an FWFT FIFO, the first word written into an empty buffer immediately "falls through" to the output pins.

- **Read-Enable as Acknowledge:** Once you read the data on the output, asserting the read-enable essentially tells the FIFO, "I am done with this word, present the next one".
- **Latency:** Standard FIFOs have a read latency of one clock cycle, whereas FWFT FIFOs present the data combinatorially on the output before a read request is even made

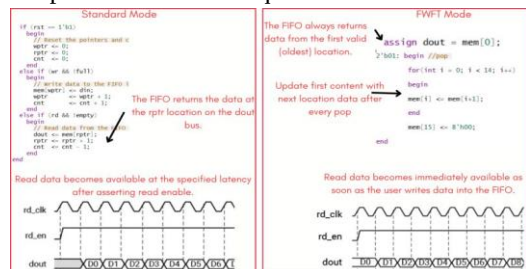


Figure 1

Latency Optimization:

- Zero-Latency Bypass (FWFT)
- Look-Ahead Word and Flag Engine
- Handshake Qualification

A. Zero-Latency Bypass(FWFT) [3]

Bypass condition (empty): if a consumer module attempts to read while the fifo is empty, a write operation on the exact same cycle routes data_in directly to data_out combinationally. The data bypasses the memory array entirely, achieving **zero-cycle latency**.

Pre-fetch condition (!empty): if data is already in the fifo, the memory array output is pre-fetched and sits waiting on the data_out bus. The consumer module does not need to assert rd_en and wait; the data is readable immediately.

B. Look ahead word and flag engine [4]

Calculating flags combinationally using arithmetic subtractors ($wr_ptr - rd_ptr$) introduces long routing delays that limit your maximum clock frequency ($\backslash(F_{\text{max}})$). This design resolves that by tracking the status sequentially inside an always block.

```
bit sample_wr; bit sample_rd; sample_wr = wr_en && !full;
sample_rd = rd_en && !empty;
```

C. Hand shake Qualification[5]

Internal variables sample_wr and sample_rd validate incoming signals.

A write only executes if wr_en is high and the FIFO is not full.

A read only executes if rd_en is high and the FIFO is not empty. This prevents corrupting data or pointers.

word_count: Keeps an exact running tally of the total packets currently resting inside the memory array. It increments on a write-only (2'b10), decrements on a read-only (2'b01), and remains unchanged during simultaneous read/write cycles or idle states.

almost_full Look-Ahead: Instead of evaluating the flag after word_count updates, this logic checks what the count **will become** on the next rising clock edge.

By pre-calculating this look-ahead value, the almost_full flag can be registered directly into a stable **flip-flop**. This completely removes combinational logic backpressure from your critical path, ensuring high-speed operation.

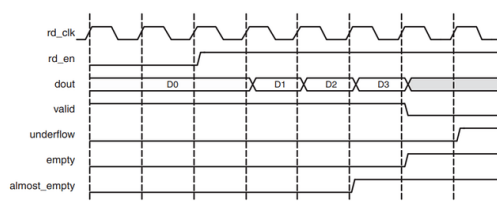


Figure 2

Unlike the standard read mode, the FWFT empty flag is asserted after the last data is read from the FIFO. When empty is asserted, VALID is deasserted.

In the standard read mode, when empty is asserted, VALID is asserted for 1 clock cycle.

The FWFT feature also increases the effective read depth of the FIFO by two read words.

The FWFT feature adds two clock cycle latency to the deassertion of empty, when the first data is written into a empty FIFO.

FWFT is useful in applications that require Low-latency access to data and to applications that require throttling based on the contents of the read data

III.PROPOSED METHOD

This project involves writing synthesizable RTL code (Verilog) for each component,. The development process includes.

In proposed the latency optimization through threshold control for FIFO is designed by using FIRST WORD FALL THROUGH(FWFT).

Almost_full parameter is used to read data even the read enable is not high.

In proposed the latency optimization through threshold control for FIFO is designed by using FIRST WORD FALL THROUGH(FWFT).

Almost_full parameter is used to read data even the read enable is not high.

The data is read immediately when the data is written after the write pointer exceeds threshold value.

IV.RESULTS AND DISCUSSIONS

RTL level verification is completed and results are shown below:

- **wr_ptr and rd_ptr:** These pointers are configured with **one extra bit** beyond the memory address space (e.g., 5 bits for a depth of 16). This extra bit acts as a **wrap-around bit** to distinguish between a completely empty FIFO and a completely full FIFO.

```
assign empty = (wr_ptr == rd_ptr);
assign full = (wr_ptr[ADDR_WIDTH] !=
rd_ptr[ADDR_WIDTH])&& (wr_ptr[ADDR_WIDTH-1:0]
== rd_ptr[ADDR_WIDTH-1:0]);
```

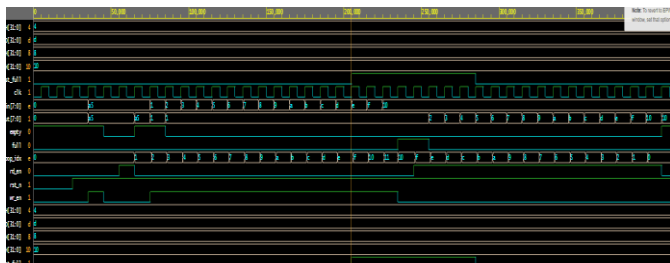


Figure 3:simulated output waveform

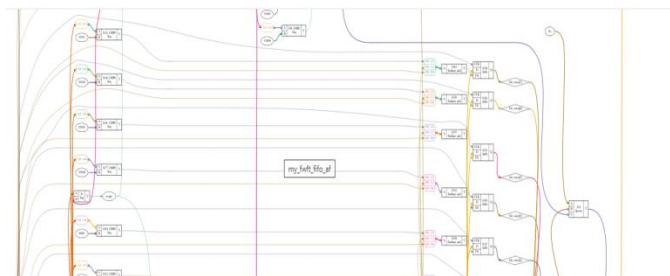


Figure 4: synthesized output

V, CONCLUSION

Parameterized synchronous FIFO with optimized latency is implemented and verified in Verilog with threshold value by using the flag almost full.

REFERENCES

- [1] V. Sklyarov, I. Skliarova, A. Barkalov, and L. Sklyarova, Handbook on Synthesis of Finite State Machines. Cham, Switzerland: Springer, 2019.

- [2] Xilinx Inc., “Vivado Design Suite User Guide: Synthesis (UG901),” Ver. 2022.2, Xilinx, 2022. [Online]. <https://docs.xilinx.com/> Available:
- [3] M. D. Ciletti, Advanced Digital Design with the Verilog HDL, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2011.
- [4] S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 2003.

Funding Declaration

The authors declare that no funds, grants, or other forms of financial support were received from any organization or institution for the conduct of this research.